

Design Document

Campus Guide — One Platform for Campus Life, Career Guidance & Peer-Mentored Academic Help

"From requirements to architecture: the complete system design of the campus-verified student platform."

Project Title	Campus Guide — Campus Info Hub, Career Path Finder & Mentor-Powered Help
Document Type	Software Design Document (Architecture, Data, UI & Behavioral Design)
Student Name	Kanij Fatema Bipa
Student ID	0112230494
Department	Department of Computer Science & Engineering
Course Name	Software Engineering Laboratory (CSE 3422)
Instructor	Rejwan Ahmed
Submission Date	26/09/2026
Companion Documents	SRS v1.0 (Campus-Guide-SRS.pdf) · Test Plan (SRS Ch. 15)
Version	1.0

This document translates the approved SRS into an implementable design: layered architecture, relational and document data models, behavioral models, interface design, and a traceable mapping from every functional requirement to its design element. Screenshot placeholders are marked throughout for images to be captured once the implementation is complete.

Table of Contents

Chapter 1 — Introduction	5
1.1 Purpose of This Document.....	5
1.2 Scope	5
1.3 Definitions and Conventions.....	5
1.4 References	5
1.5 Document Overview	5
Chapter 2 — Design Overview	6
2.1 Design Goals	6
2.2 Technology Stack and Rationale.....	6
Chapter 3 — Architectural Design	8
3.1 System Architecture	8
3.2 Component Decomposition.....	9
3.3 Deployment View	10
Chapter 4 — Data Design	11
4.1 Entity-Relationship Model	11
4.2 Relational Schema (PostgreSQL via Prisma)	11
4.3 MongoDB Collections	12
4.4 Data Validation Rules	12
Chapter 5 — Behavioral Design	13
5.1 Use-Case View	13
5.2 Activity Flows	13
5.3 Sequence: Escrow Payment.....	16
5.4 State Machines	17
Chapter 6 — User-Interface Design	18
6.1 Design Principles	18
6.2 Key Screens (Screenshot Slots).....	18
6.3 Wireframe Contracts	19
6.4 Empty, Loading and Error States	19
Chapter 7 — Interface Design	21
7.1 API Surface	21
7.2 External Interfaces	21
7.3 Context and Data Flow	22
Chapter 8 — Security Design	24
8.1 Authentication and Session Security	24
8.2 Authorization Model	24
8.3 Escrow Integrity	24
8.4 Data Protection.....	24
Chapter 9 — Requirements-to-Design Traceability	25
Chapter 10 — Design Verification	26

List of Figures

Figure 3.1 — Layered architecture of the Campus Guide platform	8
Figure 3.2 — Deployment diagram: Vercel edge/lambda runtime with managed data services	10
Figure 4.1 — ER diagram of the relational core (PostgreSQL)	11
Figure 5.1 — Use case diagram of the Campus Guide platform	13
Figure 5.2 — Activity diagram: student registration with @uiu.ac.bd verification.....	14
Figure 5.3 — Activity diagram: mentor session booking with bKash escrow, auto-confirm, and dispute branch.....	15
Figure 5.4 — Sequence diagram: booking payment via bKash with escrow hold.....	16
Figure 5.5 — State machine: booking and escrow states from payment to release or refund	17
Figure 7.1 — Context diagram: the platform and its external actors	22
Figure 7.2 — Level-0 data-flow diagram across the three pillars	23

List of Tables

Table 2.1 — Technology stack of the Campus Guide platform	6
Table 3.1 — Core components and their SRS responsibilities	9
Table 4.1 — Principal relations and constraints	11
Table 6.1 — Wireframe inventory (full list in SRS Appendix E)	19
Table 7.1 — External services and failure policy.....	21
Table 9.1 — Functional requirement areas mapped to design elements	25

Chapter 1 — Introduction

1.1 Purpose of This Document

This Design Document describes the software design of the **Campus Guide** platform as derived from the approved Software Requirements Specification (SRS v1.0). It fixes the system architecture, the database structure, the behavioral models, the API surface, and the user-interface design before implementation begins, so that every module built by the team can be traced back to a documented requirement.

1.2 Scope

The document covers the design of all three product pillars — the **Campus Info Hub**, the **Career Path Finder**, and **Mentor-Powered Help** with bKash escrow — together with the shared foundation of campus-verified accounts, role-based access control, notifications, and administration. Design decisions already fixed by the SRS (Next.js full-stack, PostgreSQL + MongoDB, Vercel hosting) are assumed and are not re-litigated here.

1.3 Definitions and Conventions

All requirement identifiers (FR-xx-nn, NFR-xx-nn, UC-nn) refer to the SRS. Diagrams follow UML conventions; data-flow diagrams are Yourdon-style. Wherever a figure or table duplicates the SRS, this document references the SRS rather than repeating it, and adds only design-level detail.

1.4 References

- Campus Guide — Software Requirements Specification, v1.0 (companion document).
- IEEE 29148-2018, ISO/IEC/IEEE Standard for the Engineering of Systems — Requirements
- Next.js 14 (App Router) documentation — <https://nextjs.org/docs>
- Prisma ORM documentation — <https://www.prisma.io/docs>
- bKash Merchant API / Tokenized Checkout documentation

1.5 Document Overview

Chapter 2 states the design goals and explains the technology stack. Chapter 3 presents the architectural design with its deployment view. Chapter 4 details the data design (ER model, relational schema, MongoDB collections). Chapter 5 contains the behavioral design: use-case, activity, sequence, and state-machine views. Chapter 6 specifies the user-interface design and reserves screenshot slots. Chapter 7 documents the API and external interfaces. Chapter 8 covers the security design. Chapter 9 maps every functional requirement to its design elements, and Chapter 10 summarizes how the design will be verified by testing.

Chapter 2 — Design Overview

2.1 Design Goals

1. **Traceability.** Every SRS functional requirement maps to at least one component, database table, and API route (Chapter 9).
2. **Verified-by-design trust.** The @uii.ac.bd verification, escrow accounting, and dispute handling are enforced by schema constraints and state machines, not by UI conventions alone.
3. **One platform, one session.** All three pillars share one account, one role model, and one notification pipeline (JWT in httpOnly cookies).
4. **Thin client, server-owned rules.** Business rules (eligibility, commission, refund windows) execute in Next.js server actions / route handlers so they cannot be bypassed from the browser.
5. **Prepared for growth.** The schema is multi-tenant-ready (Institution table) even though only UIU is configured at launch.

2.2 Technology Stack and Rationale

Table 2.1 — Technology stack of the Campus Guide platform

Layer	Technology	Design Rationale
Frontend	Next.js 14 (App Router), React Server Components, Tailwind CSS	One framework renders both marketing pages (SSR for SEO) and the app shell; server components cut client-side JavaScript for the feed-heavy Info Hub.
Backend	Next.js API routes / server actions	Colocated with the frontend — one repository, one deploy, typed contracts shared directly (no separate REST layer to drift).
Relational store	PostgreSQL (Neon) + Prisma ORM	Escrow money, bookings, and identities demand transactions and joins; Prisma gives typed queries and migrations reviewed in Git.
Document store	MongoDB Atlas	High-volume, read-only collections (event feed, career resources, scraped job posts) tolerate schema flexibility and are cheap to scale independently.
Cache	Redis (Upstash)	Session tokens, rate-limit counters, and hot feed caching with TTL eviction.
Payments	bKash Tokenized Checkout (Merchant API)	Dominant campus payment rail in Bangladesh; tokenized flow keeps card/wallet credentials outside our perimeter.
Email	Resend / SMTP	Verification links and digests; provider-abstracted behind one module so it can be swapped.
Hosting	Vercel (frontend + API) · Neon/Atlas managed DBs	Zero-ops deployment from the same Next.js repo; managed databases include backups and failover.

Design at a Glance	
Architecture	Next.js full-stack — RSC frontend, server-action backend, PostgreSQL + MongoDB, Redis cache
Design pattern	Route handlers own business rules; Prisma schema is the single source of data truth
Security posture	Campus-verified accounts, RBAC, httpOnly JWT sessions, append-only escrow ledger
This document	10 chapters, 12 figures, full FR-to-design traceability

Chapter 3 — Architectural Design

3.1 System Architecture

The platform is a single Next.js application with a clean separation between rendering (React Server Components), application services (route handlers / server actions), and persistence (PostgreSQL via Prisma, MongoDB, Redis). There is no separately deployed backend; the API surface lives in the same repository and deployment unit, which removes contract drift and simplifies CI.

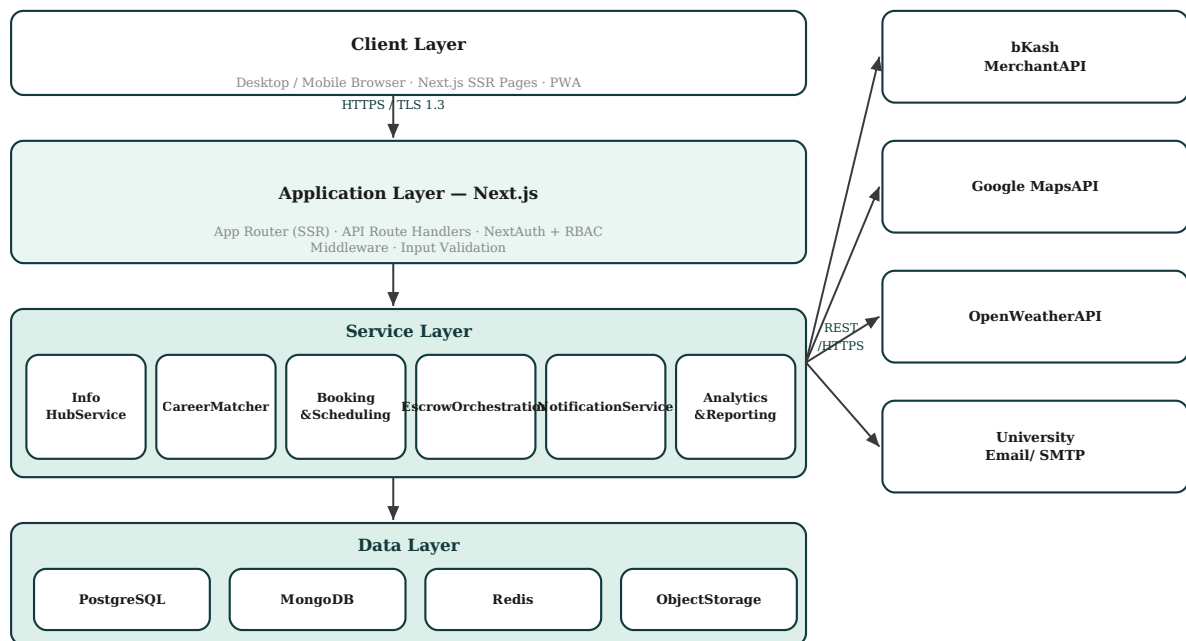


Figure 3.1 — Layered architecture of the Campus Guide platform

Layer responsibilities. The presentation layer renders role-aware UI from server components and never formats money or decides eligibility. The application layer validates all input with Zod schemas, enforces the business rules of the SRS (verification windows, escrow states, dispute deadlines), and is the only layer allowed to open transactions against

PostgreSQL. The data-access layer is Prisma models plus typed MongoDB collection helpers; it contains no business logic. The integration layer isolates external services (bKash, email, Google Maps embed, optional calendar sync) behind adapter interfaces so tests can stub them.

3.2 Component Decomposition

Table 3.1 — Core components and their SRS responsibilities

Component	Responsibility	Owning layer	Key SRS refs
auth-service	Signup gating, verification tokens, session issue/refresh, password hashing (bcrypt), logout	Application	FR-AU-01...09
profile-service	Student & mentor profiles, portfolio items, availability slots	Application	FR-PM-01...08
info-hub-service	Event feed, announcement ingestion, club posts, map data, YouTube sync	Application + Mongo	FR-CI-01...10
career-service	Skill assessment, roadmap engine, resource matcher, job-post mirror	Application + Mongo	FR-CP-01...09
mentor-market-service	Mentor search/filter, booking, session lifecycle	Application	FR-MH-01...09
escrow-service	bKash charge, hold, split, refund; append-only ledger writes	Application	FR-MH-02, 06, 08; NFR-SE-09
notify-service	In-app + email notifications, digest scheduler, preferences	Application	FR-NT-01...05
report-service	Usage reports, admin dashboards, CSV export	Application	FR-RP-01...04
admin-service	User moderation, event approval, dispute ruling, ban/suspend	Application	FR-AD-01...06

3.3 Deployment View

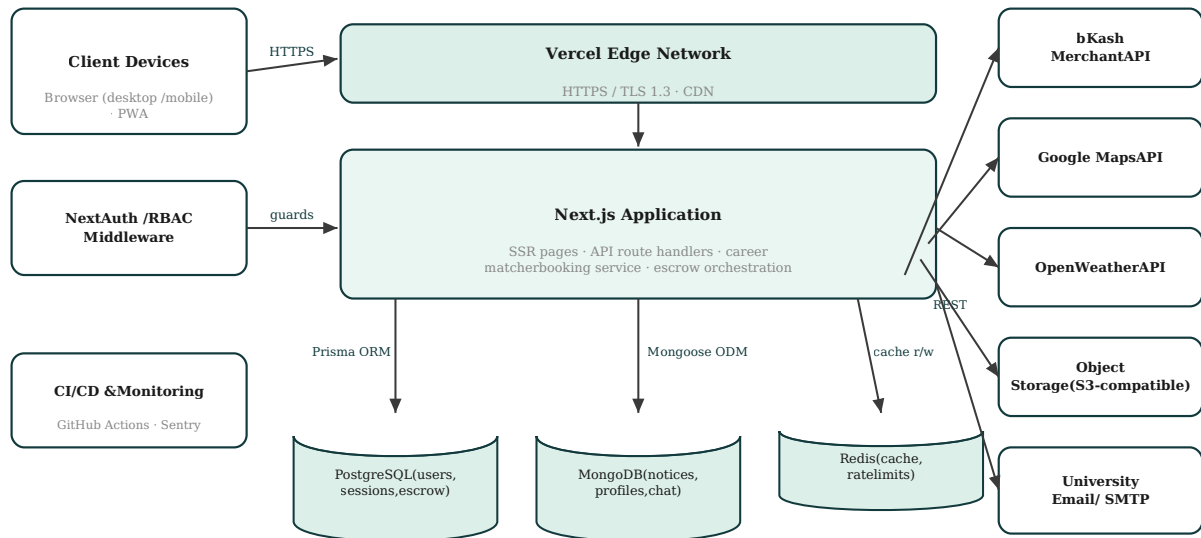


Figure 3.2 — Deployment diagram: Vercel edge/lambda runtime with managed data services

All compute runs on Vercel: static assets on the CDN, server components and route handlers on serverless functions per region. PostgreSQL (Neon) and MongoDB Atlas run as managed services with automated backups; Redis (Upstash) is accessed over TLS. This yields the NFR-SC-01 availability target (99.5%) without any self-managed servers, and lets the team scale reads through caching before writes ever become a bottleneck.

Chapter 4 — Data Design

4.1 Entity-Relationship Model

The relational core keeps the entities whose integrity is non-negotiable: users, profiles, bookings, escrow ledger entries, ratings, and disputes. The ER model below is identical to SRS §11.5 and is reproduced here as the authority for the Prisma schema in §4.2.

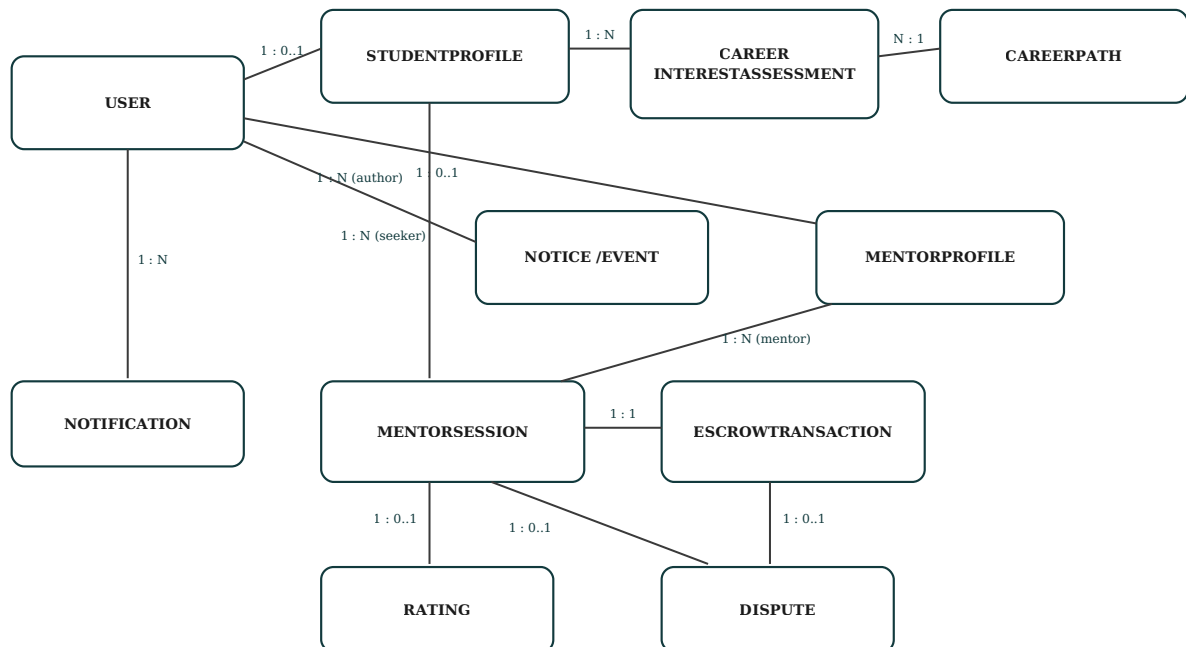


Figure 4.1 — ER diagram of the relational core (PostgreSQL)

4.2 Relational Schema (PostgreSQL via Prisma)

Key tables and their relationships are summarized below; the complete Prisma schema is in SRS Appendix C and will be committed as `prisma/schema.prisma`.

Table 4.1 — Principal relations and constraints

Relation	Purpose and key columns	Integrity constraints
User	Identity: email (unique, @uiu.ac.bd), passwordHash, role enum (STUDENT MENTOR FACULTY ADMIN), status enum	Verified accounts only get role assignment; deletes are soft (status=CLOSED)
StudentProfile / MentorProfile	1:1 extensions with department, intake, skills[], ratings aggregates	ON DELETE CASCADE from User; mentor rating updated transactionally with Rating
AvailabilitySlot	Mentor time windows (start, end, recurring flag)	No overlapping slots per mentor (app-level check in transaction)
Booking	Session: studentId, mentorId, slot, subject, fileUrl, status enum	Status transitions restricted by state machine (Fig. 5.4)
EscrowLedger	Append-only rows: bookingId, kind (HOLD RELEASE REFUND COMMISSION), amount, balanceAfter	INSERT-only grant; updates forbidden at DB level (NFR-SE-09)
Rating	1:1 with completed Booking; stars 1-5, comment	Unique per booking; only BOOKED+COMPLETED sessions may be rated
Dispute	bookingId, evidenceUrl, status, adminRuling	Freezes booking state machine; 7-day window enforced by cron check
Institution	Multi-tenant root: domain suffix, name, isActive	Only UIU row active at launch; others dormant

4.3 MongoDB Collections

Three document collections hold high-volume, read-mostly data where fixed schemas would slow iteration without buying integrity: **events** (feed items with denormalized club data and TTL-ed soft deletes), **career_resources** (roadmap steps, guides, scraped job posts with the raw source payload preserved), and **notifications_cache** (fan-out copies of recent notifications per user). Documents carry `sourceUrl` and `scrapedAt` so any ingestion bug can be replayed or purged by query.

4.4 Data Validation Rules

- Emails must match the institution domain suffix; rejected before any DB write (FR-UM-01).
- All money values are `DECIMAL(10,2)` in BDT; the ledger recomputes balanceAfter server-side on every insert.
- Dates are stored in UTC; rendering converts to Asia/Dhaka.
- Slot bookings recheck mentor availability inside the same transaction that inserts the Booking row.

Chapter 5 — Behavioral Design

5.1 Use-Case View

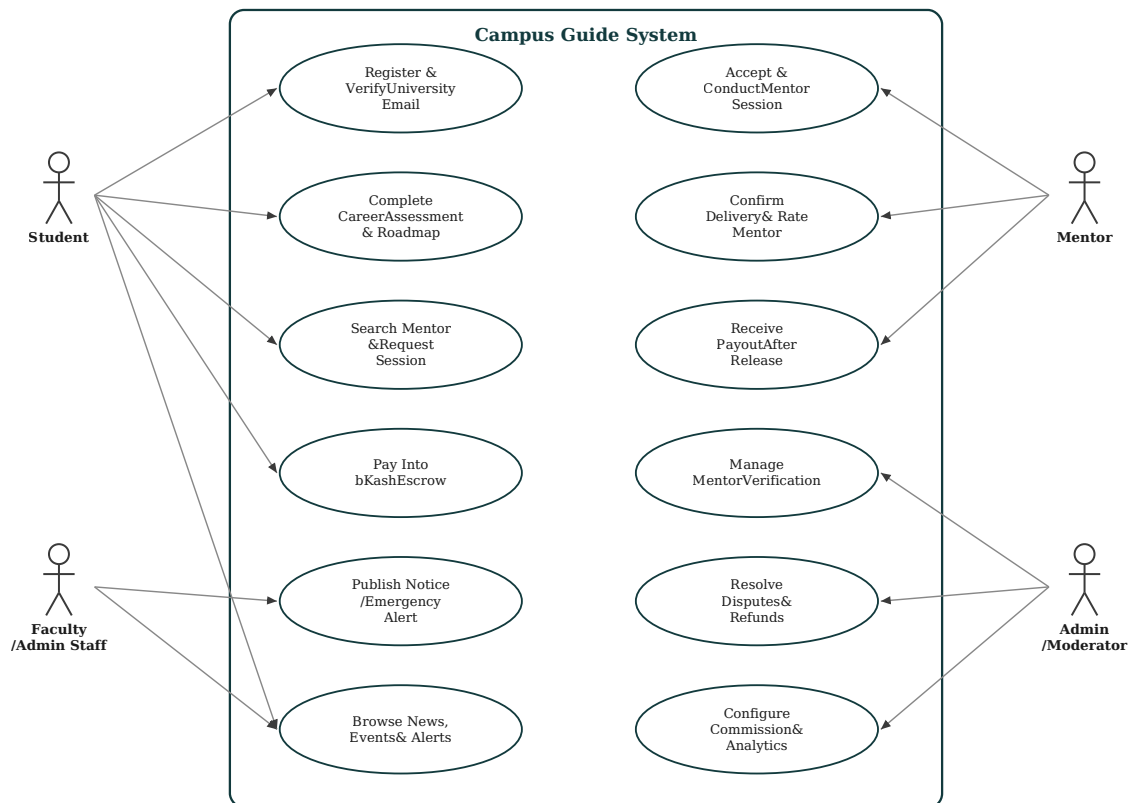


Figure 5.1 — Use case diagram of the Campus Guide platform

The twelve SRS use cases (UC-01...UC-12) are unchanged by design; this section adds only the flows that require sequencing guarantees.

5.2 Activity Flows

Two flows carry the highest correctness risk and are therefore modeled as activity diagrams: registration with university verification, and the mentor-session escrow lifecycle.

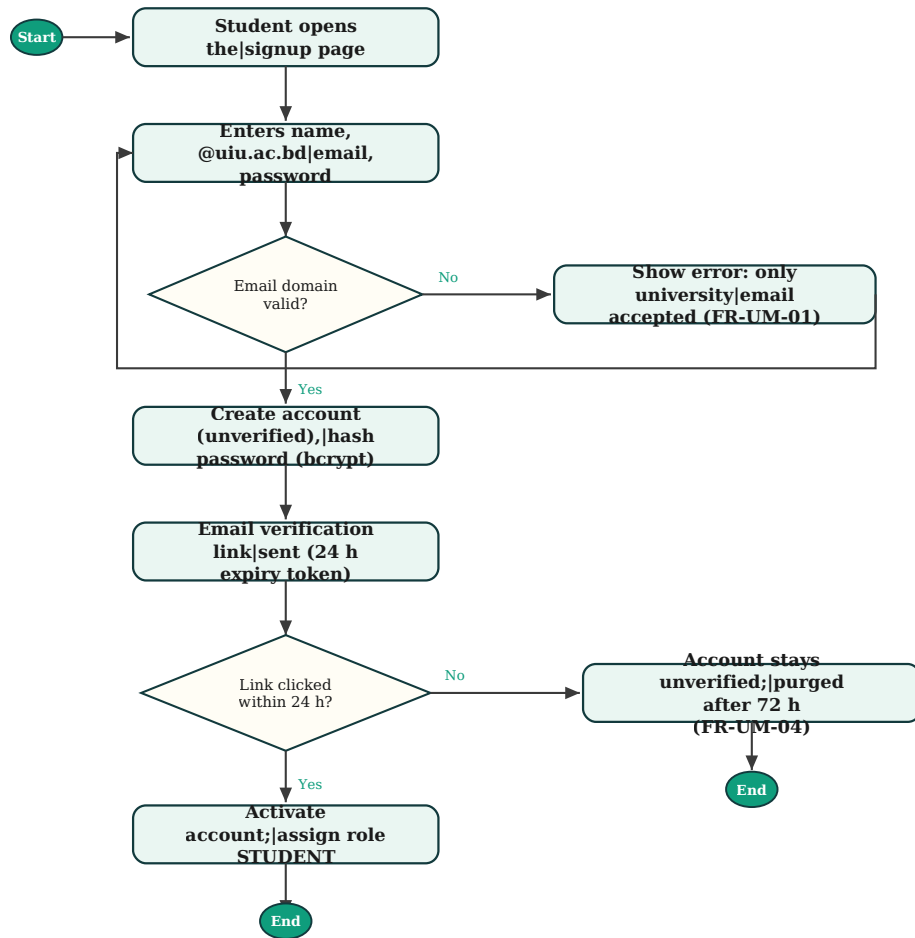


Figure 5.2 — Activity diagram: student registration with @uiu.ac.bd verification

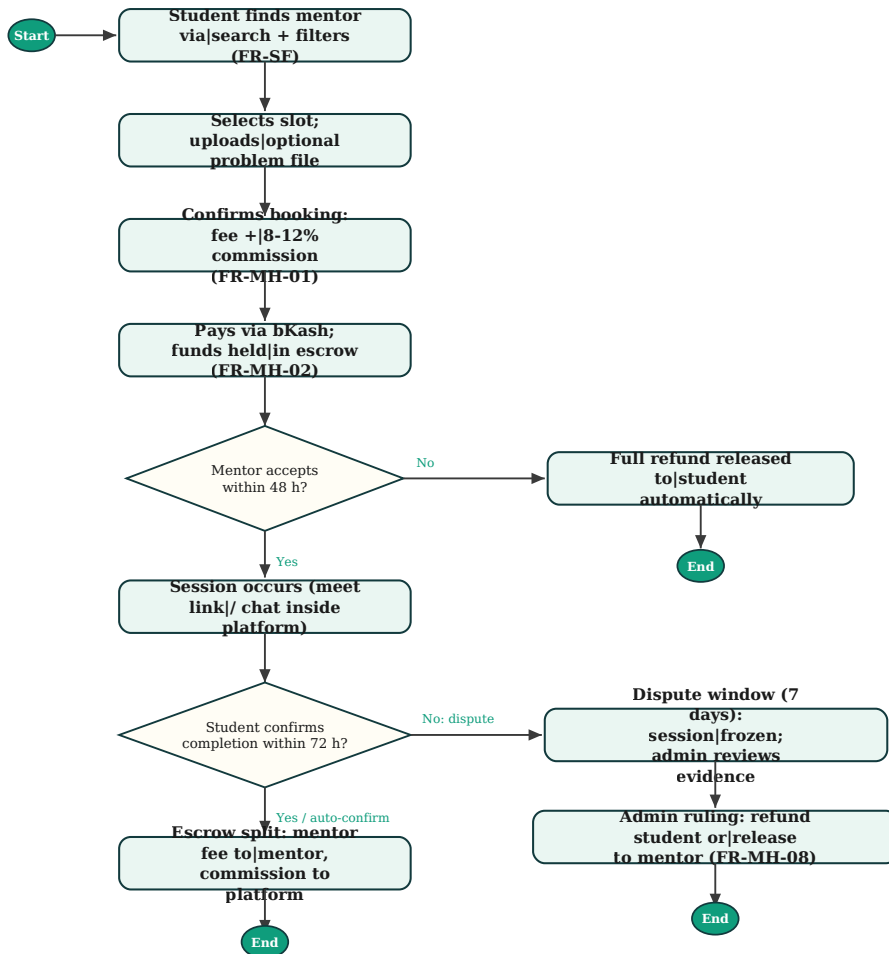


Figure 5.3 — Activity diagram: mentor session booking with bKash escrow, auto-confirm, and dispute branch

5.3 Sequence: Escrow Payment

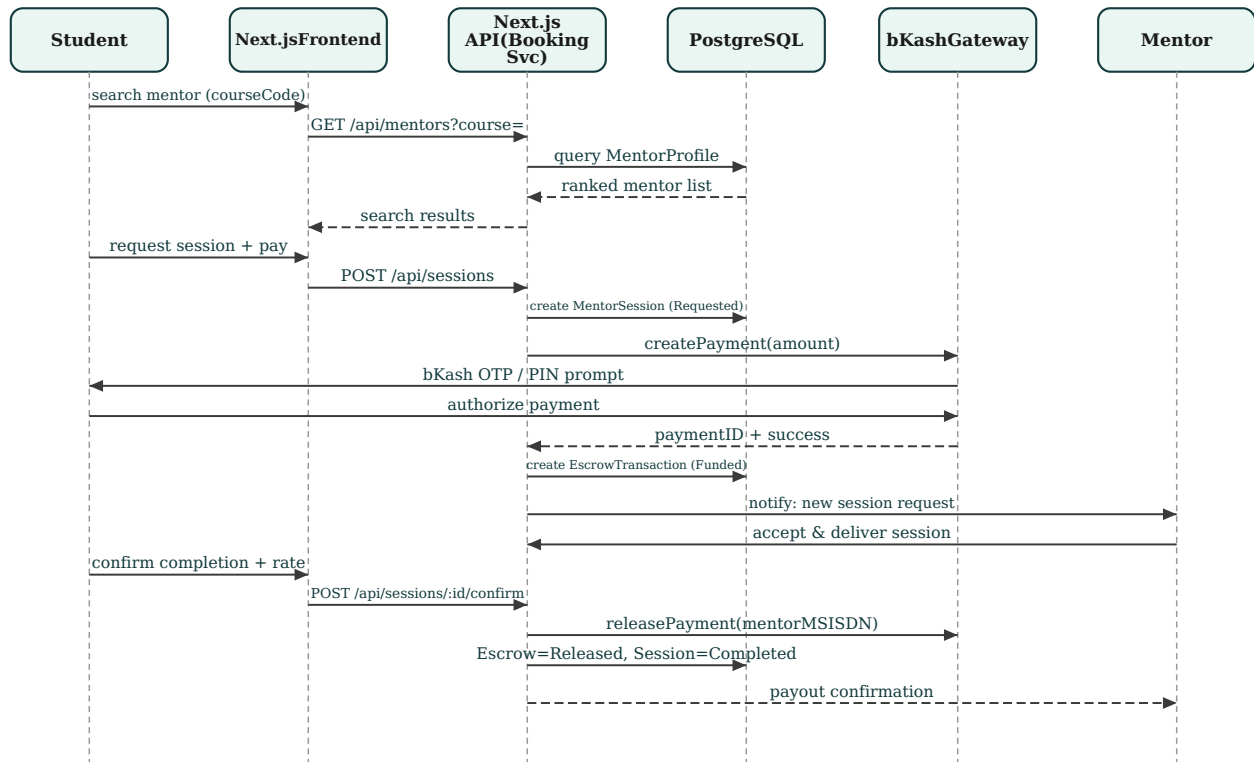


Figure 5.4 — Sequence diagram: booking payment via bKash with escrow hold

5.4 State Machines

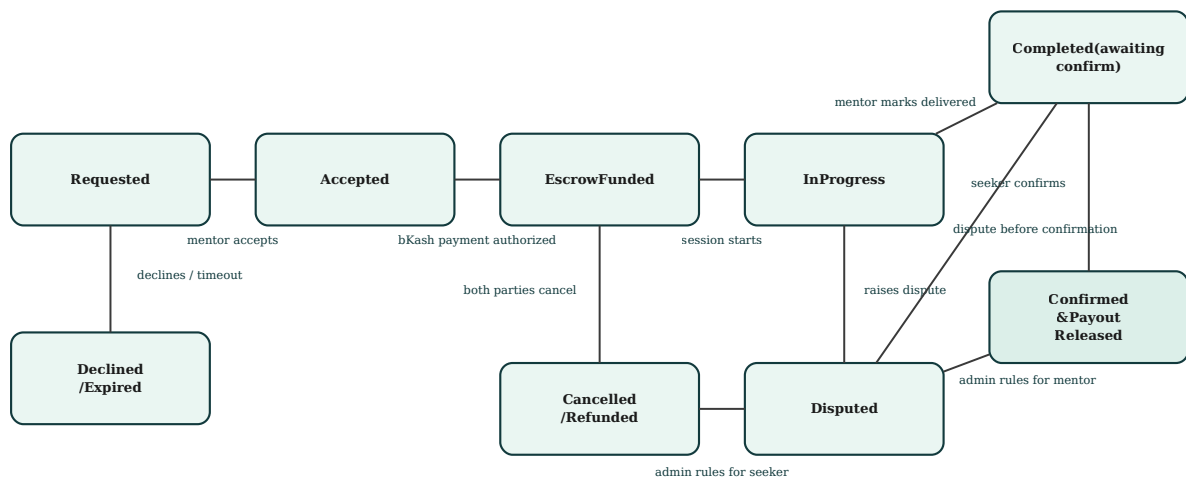


Figure 5.5 — State machine: booking and escrow states from payment to release or refund

Both state machines are enforced by a single `transition()` helper in `mentor-market-service` that validates `(currentState, event)` pairs against a table before mutating; illegal transitions log a 422 and change nothing. The dispute freeze is modeled as a guarded state, never a flag column, so a frozen booking physically cannot confirm.

Chapter 6 — User-Interface Design

6.1 Design Principles

- **Mobile-first, between-classes context.** Primary actions reachable one-handed; feed items render fully at 360 px width.
- **Trust surfaces.** Escrow states, amounts, and deadlines are always visible verbatim — never hidden behind tooltips.
- **Consistent states.** Every list has designed empty, loading, and error states (§6.4).
- **Accessible by default.** WCAG 2.1 AA contrast, keyboard navigation, visible focus rings, Arabic-free Bangla/English typography at 14 px minimum (NFR-AC-01...05).

6.2 Key Screens (Screenshot Slots)

The screens below are specified as wireframe contracts; each slot reserves space for the screenshot to be captured at implementation time. Captures should be taken at 1440×900 (desktop) and 390×844 (mobile).

SCREENSHOT SLOT 6.1

Home / Campus Info Hub feed

Show: announcement banner, filter chips (All | Academic | Clubs | Deadlines), event cards with date blocks, and the bottom nav (Home, Career, Mentors, Profile). Capture both an empty state and a populated state.

SCREENSHOT SLOT 6.2

Career Path Finder — assessment & roadmap

Show: skill-assessment form (one question per screen with progress bar) and the resulting roadmap view with resource cards grouped by stage, each with the estimated-time chip.

SCREENSHOT SLOT 6.3

Mentor marketplace

Show: search bar, department/skill/rating filters, mentor cards (photo, skills, rating, per-session fee in BDT), and the availability calendar of one selected mentor.

SCREENSHOT SLOT 6.4**Booking & bKash escrow checkout**

Show: booking summary (fee + commission total), problem-file upload, the bKash tokenized-checkout redirect screen, and the resulting booking detail page with escrow status badge (PENDING → HELD → RELEASED/REFUNDED).

SCREENSHOT SLOT 6.5**Session room & rating flow**

Show: in-platform session view (meet link + chat), the completion confirmation prompt (shown at 72 h with countdown), and the 5-star rating form after release.

SCREENSHOT SLOT 6.6**Admin console**

Show: dispute queue with evidence viewer, user management with ban/suspend actions and audit trail, and the events-approval moderation screen.

6.3 Wireframe Contracts

Table 6.1 — Wireframe inventory (full list in SRS Appendix E)

Screen	Primary components	Primary actions / FRs
Auth screens	Signup, verify-email, login, forgot/reset	FR-AU-01...09
Info Hub home	Feed, filters, event detail, map embed	FR-CI-01...07
Career workspace	Assessment wizard, roadmap, resources, job mirror	FR-CP-01...09
Mentor marketplace	Search, filters, mentor profile, calendar, checkout	FR-SF-01...06, FR-MH-01...05
Session & ratings	Session room, confirm, rate, dispute form	FR-MH-06...09
Profile & settings	Profile editor, portfolio, notifications prefs	FR-PM-01...08, FR-NT-05
Admin console	Moderation, disputes, reports	FR-AD-01...06, FR-RP-01...04

6.4 Empty, Loading and Error States

Every list view implements: (a) an empty state naming the next user action, (b) skeleton loaders for the first paint only, and (c) an error state with a retry affordance. Escrow-

affected screens additionally show the deadline countdown (48 h accept, 72 h confirm, 7-day dispute) directly in the status card, satisfying NFR-US-04 (system status visibility).

Chapter 7 — Interface Design

7.1 API Surface

The full route table with request/response contracts is specified in SRS Appendix F; the design-level API rules are fixed here.

- All mutating endpoints are POST/PUT with Zod-validated JSON bodies; validation errors return 422 with a field-level machine-readable list.
- Money is always exchanged as integer poisha (1 BDT = 100 poisha) to avoid float drift; only the presentation layer formats BDT.
- Auth is a JWT in an httpOnly, Secure, SameSite=Lax cookie, refreshed on activity with a 7-day idle timeout (FR-AU-05).
- Every route enforces role guards at the handler top before any DB access (NFR-SE-02).

7.2 External Interfaces

Table 7.1 — External services and failure policy

Service	Used for	Failure policy
bKash Tokenized Checkout	Charge, refund, release (escrow)	Idempotency keys per operation; retries with exponential backoff; failures land booking in PAYMENT_FAILED with a retry affordance
Email (Resend/SMTP)	Verification, digests, receipts	Queue with retry; verification email failures surface a resend button
Google Maps embed	Campus event locations	Graceful degradation to a text address if embed fails
YouTube Data API	Official channels sync (Info Hub)	Scheduled sync with delta cursor; API errors logged, feed unaffected
Job-post sources	Career job mirror	Scrapers store raw payloads; dedupe by sourceUrl+hash

7.3 Context and Data Flow

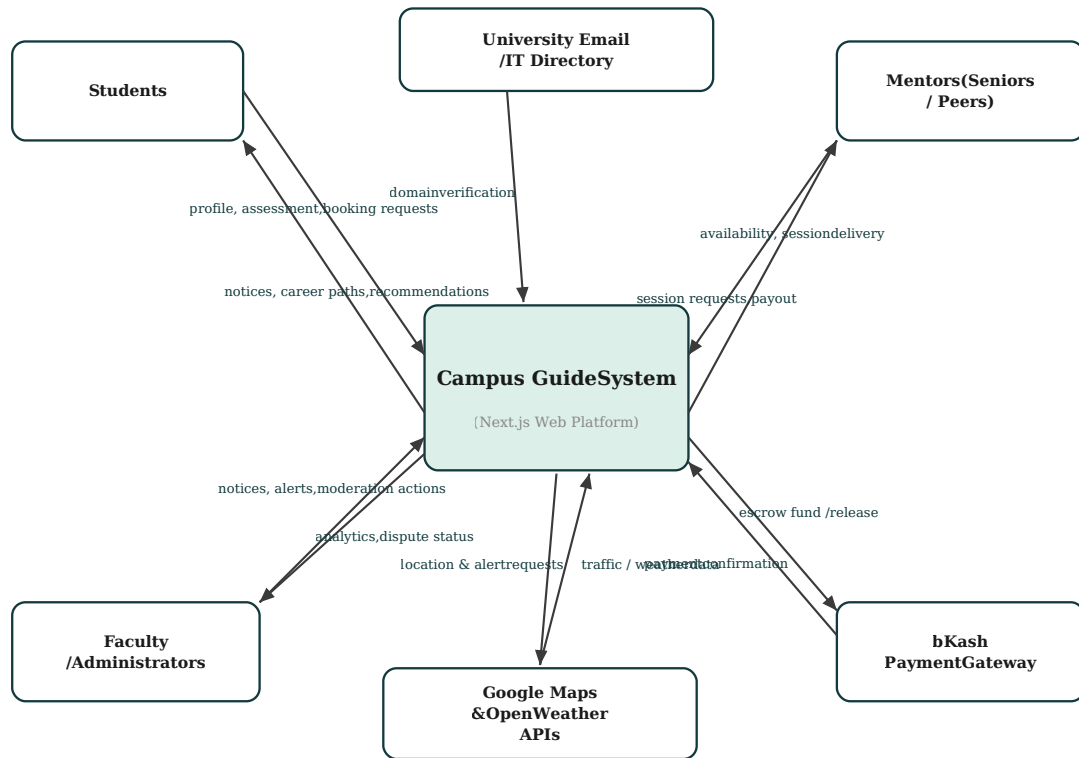


Figure 7.1 — Context diagram: the platform and its external actors

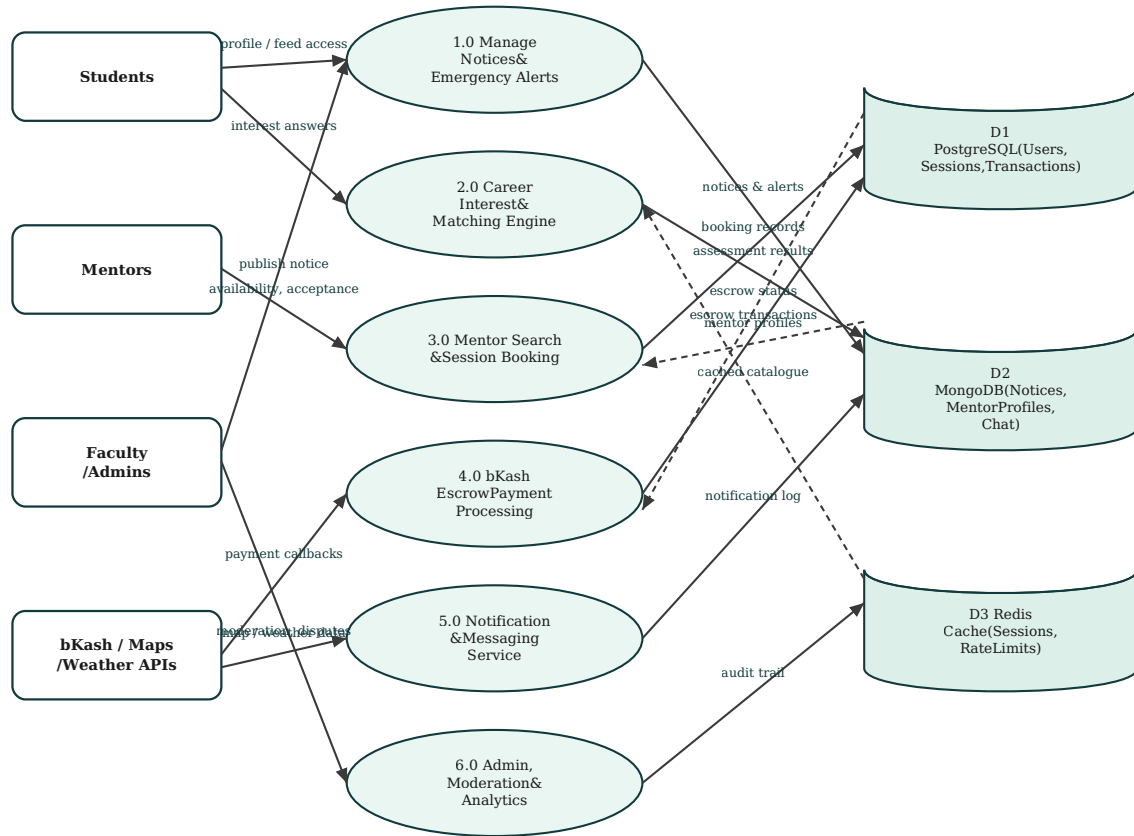


Figure 7.2 — Level-0 data-flow diagram across the three pillars

Chapter 8 — Security Design

8.1 Authentication and Session Security

- Passwords hashed with bcrypt (cost 12); no plaintext or reversible storage anywhere.
- Verification tokens are single-use, 24-hour, random 32-byte URLs; account purge at 72 h if unverified.
- JWT sessions in httpOnly cookies; refresh on activity; idle timeout 7 days; logout revokes the Redis session record so a stolen cookie dies with logout.
- Login rate limiting: 5 attempts per 15 minutes per account+IP (NFR-SE-05).

8.2 Authorization Model

Roles are STUDENT, MENTOR, FACULTY, ADMIN. Route guards, not UI hiding, define the boundary: a mentor is a student with an approved MentorProfile, so role checks are attribute-based (hasApprovedMentorProfile) wherever the SRS says "verified mentor" (FR-PM-05). Admin actions (FR-AD-01...06) require role=ADMIN and write to the append-only audit log with actor, target, reason, and timestamp.

8.3 Escrow Integrity

- Every money movement is one ledger row; the ledger balance is recomputed from the full row set nightly and alerted on mismatch.
- HOLD → RELEASE/REFUND transitions require the state machine of Fig. 5.5; amounts are cross-checked against the booking fee and commission at write time.
- Dispute freezes stop the clock: while a dispute is OPEN, no auto-confirm or release can execute, enforced in the same transaction.

8.4 Data Protection

PII is limited to name, university email, and profile fields. Data at rest is covered by the managed providers' encryption; secrets live only in Vercel encrypted environment variables. Exports (FR-RP-04) strip emails by default. Delete requests soft-close the account and schedule hard anonymization after 30 days (NFR-PV-01...05).

Chapter 9 — Requirements-to-Design Traceability

Every functional requirement area maps to components, schema objects, API groups, and behavioral models. The complete requirement-level matrix is SRS Table 16.1; the area-level design view is:

Table 9.1 — Functional requirement areas mapped to design elements

SRS area	Components	Data objects	API group	Behavior model
FR-UM / FR-AU	auth-service	User, Institution	/api/auth/*	Fig. 5.2 activity
FR-PM	profile-service	StudentProfile, MentorProfile, PortfolioItem	/api/profile/*	—
FR-CI	info-hub-service	Mongo events, EventApproval	/api/hub/*	Fig. 7.2 DFD
FR-CP	career-service	Assessment, Roadmap; Mongo career_resources	/api/career/*	Fig. 7.2 DFD
FR-MH	mentor-market-service, escrow-service	Booking, EscrowLedger, Rating, Dispute	/api/mentors/*, /api/escrow/*	Fig. 5.3 activity, Fig. 5.4 sequence, Fig. 5.5 state
FR-SF	search module (shared)	Indexes on Mongo + PG	/api/search	—
FR-NT	notify-service	Notification; Mongo cache	/api/notifications/*	—
FR-RP / FR-AD	report-service, admin-service	AuditLog, ReportRun	/api/admin/*, /api/reports/*	—
FR-EH	shared middleware	—	all routes	§7.1 error contract

Coverage rule: a requirement area is design-complete when each of its SRS requirements appears in at least one component and one test-case group. Chapter 10 records how that completeness is verified.

Chapter 10 — Design Verification

The design is verified against the SRS in three passes, all specified in SRS Chapter 15 and summarized here so reviewers can connect design to tests:

1. **Unit level.** Each service's rule functions (state machine transitions, ledger inserts, eligibility checks) are tested with boundary inputs; e.g., the escrow split function must reproduce the fee/commission arithmetic of FR-MH-01 for fees 100–10,000 BDT.
2. **Integration level.** Booking + escrow + notification run as one flow in tests with a stubbed bKash adapter, asserting the exact ledger rows and notification emissions for accept, timeout, and dispute paths.
3. **System & acceptance.** SRS §15.5 scenarios are executed against a production-like deployment; acceptance is the SRS acceptance criteria list, every one of which names a testable threshold.

Defects found during verification are logged with severity, the failing requirement, and the design element at fault, feeding the revision history of both this document and the SRS.